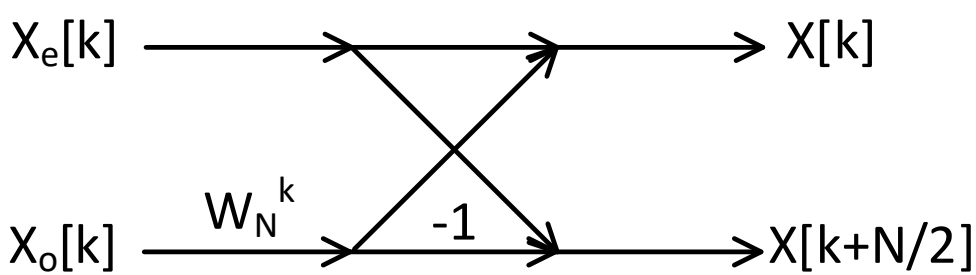
Fast Fourier Transform (FFT)

Implementation

Ira A. Fulton Schools of Engineering
Arizona State University

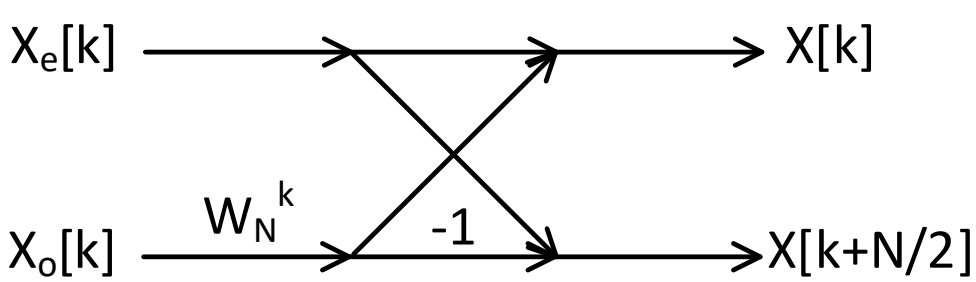
N-point FFT

- $N = 2^{\alpha}$
- We need $\alpha = \log_2 N$ stages and N/2 butterflies per stage.
- Recall butterfly equations:
  - $X[k] = X_e[k] + W_N^k X_o[k], k = 0, 1, \ldots, \frac{N}{2} - 1$
  - $X\left[k + \frac{N}{2}\right] = X_e[k] - W_N^k X_o[k], k = 0, 1, \ldots, \frac{N}{2} - 1$



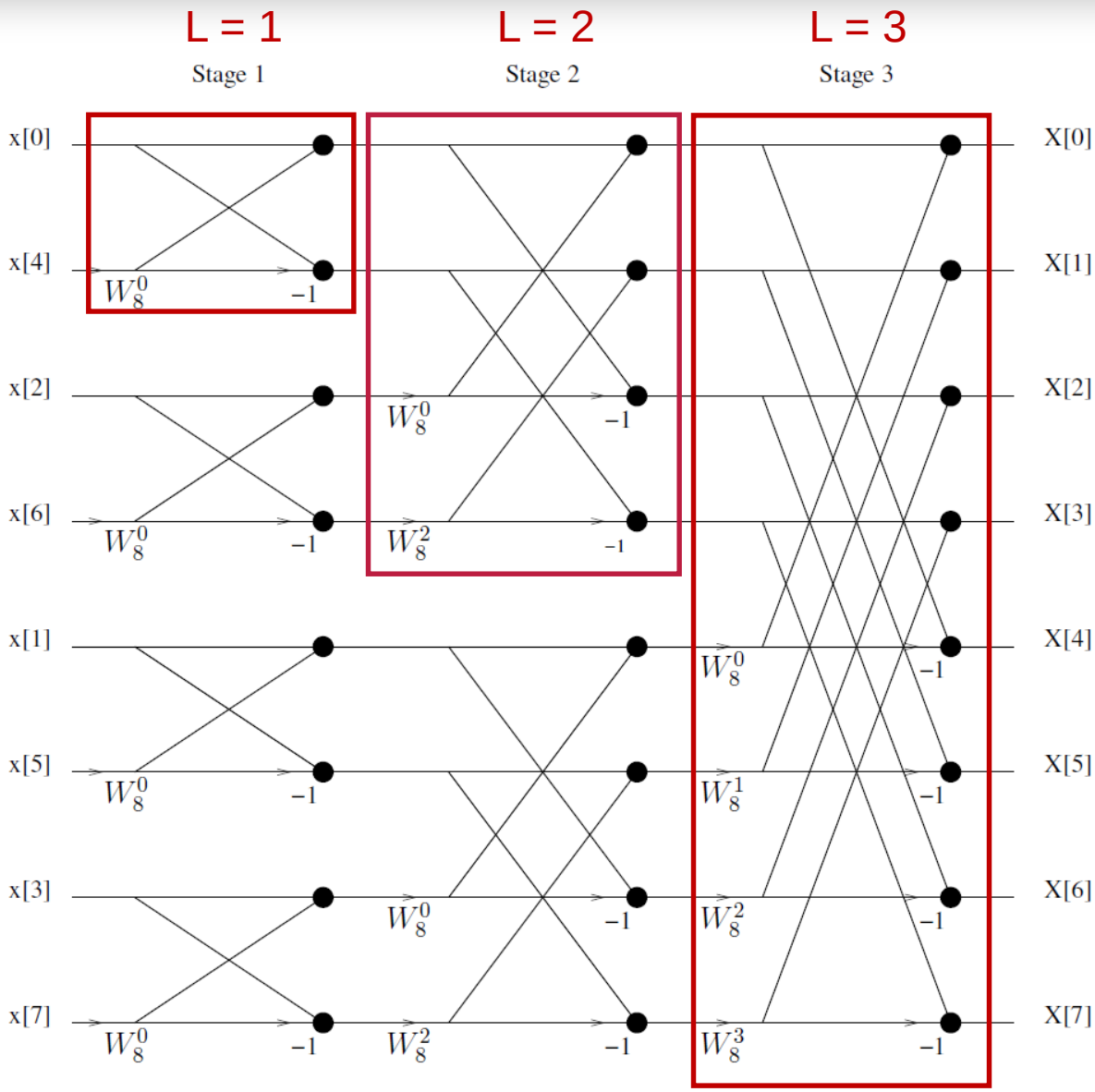
Twiddle Factor

- $W_N^k = e^{-j\frac{2\pi}{N}k}$
- **N/2 twiddle factors are needed to compute N-point FFT, i.e.** $W_N^0, W_N^1, \dots, W_N^{\frac{N}{2}-1}$.
- **Precompute α = log$_2$N twiddle factors** $W = [W_N^{\frac{N}{2}}, W_N^{\frac{N}{4}}, \dots, W_N^1]$.
- **Compute the rest of twiddle factors on the fly.**
- **Example: for N = 8, precompute** $W = [W_8^4, W_8^2, W_8^1]$.



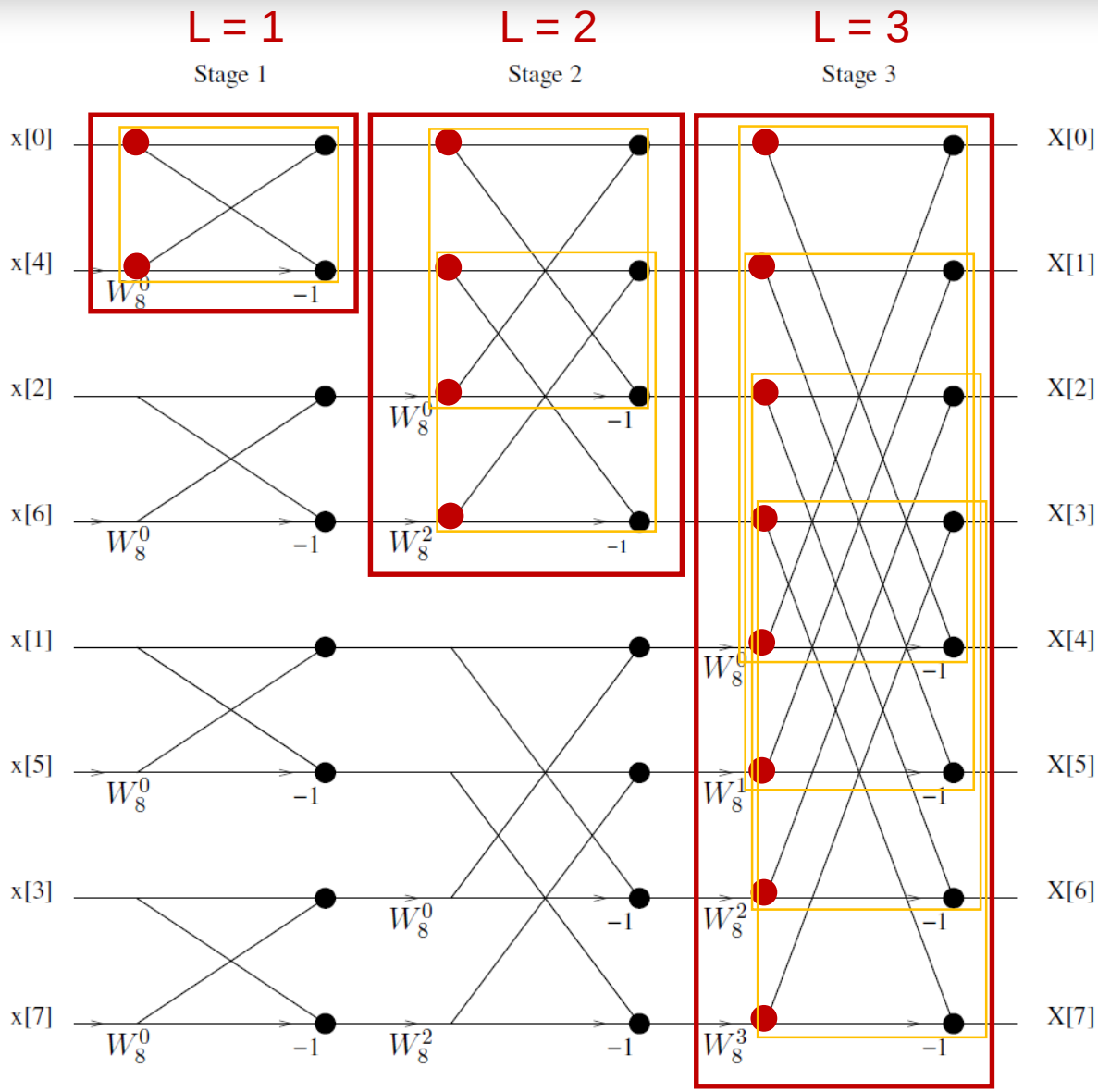
Butterfly Structure at Different Stages

- L = current stage
- D = number of points in the sub DFT butterfly structure at stage L
  - = 2^L = 1 << L
- B = number of butterflies in the sub DFT butterfly structure at stage L
  - = D/2 = D >> 1
- Exercise: compute D and B for N = 8-pt FFT for all stages of L.



Butterfly Structure at Different Stages

- D = number of points in the sub DFT butterfly structure at stage L
- B = number of butterflies in the sub DFT butterfly structure at stage L
- L = 1
  - $D = 2^L = 1 << L = 2$ (2-pt DFT)
  - $B = D/2 = D >> 1 = 1$ (1 butterfly in 2-pt DFT)
  - Repeat for 4 structures
- L = 2
  - $D = 2^L = 1 << L = 4$ (4-pt DFT)
  - $B = D/2 = D >> 1 = 2$ (2 butterflies in 4-pt DFT)
  - Repeat for 2 structures
- L = 3
  - $D = 2^L = 1 << L = 8$ (8-pt DFT)
  - $B = D/2 = D >> 1 = 4$ (4 butterflies in 8-pt DFT)
  - Repeat for 1 structure



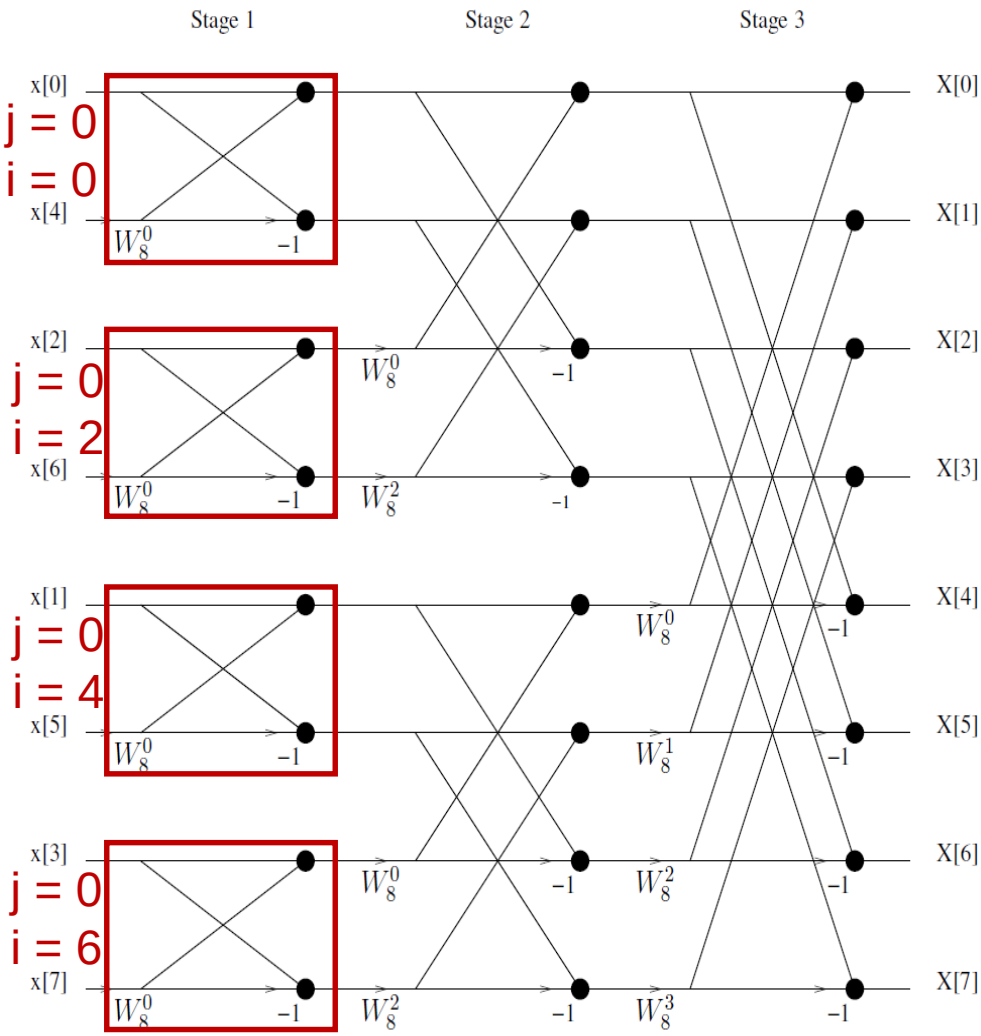
Butterfly Structure (L = 1)

```
// L is the current stage
for (L = 1; L <= □; L++)
{
   // D = number of points in the sub DFT
   // butterfly structure at stage L
   D = 1 << L; // D = 2
   // B = number of butterflies in the sub DFT
   // butterfly structure at stage L
   B = D >> 1; // B = 1
   // reset twiddle factor U = W_N^0 = 1

   // j is the butterfly index in the sub DFT
   // butterfly structure
   for (j = 0; j < □; j++) // run 1 time
   {
      // i is butterfly top line index
      for (i = j; i < N; i += □) // run 4 times
      {
         //compute butterfly bottom line index k
         k = i + □;

         // compute butterfly outputs X[i], X[k]
         // X[i] = X[i] + U*X[k]
         // X[k] = X[i] – U*X[k]
      }
      // update twiddle factor U = U*W[L-1]
   }
}
```

L = 1



Reset U = W_8^0 = 1

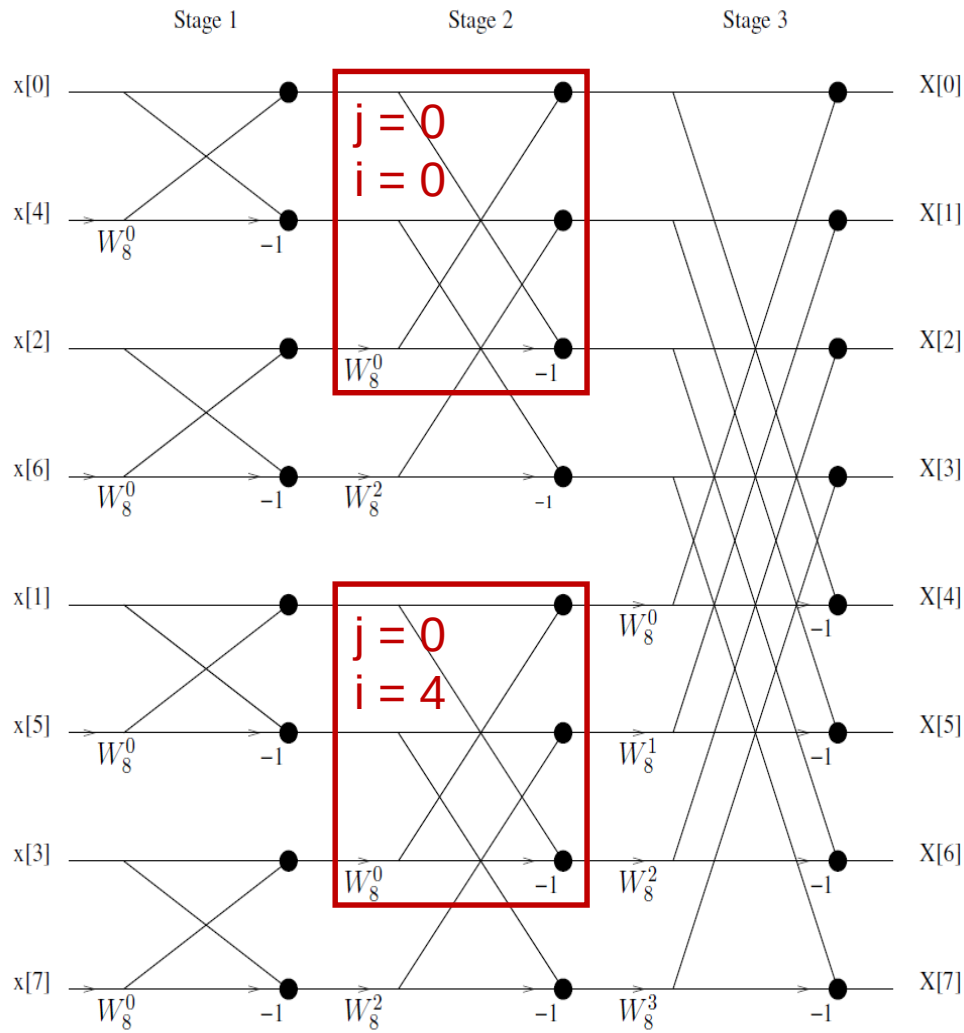
Butterfly Structure (L = 2)

```
// L is the current stage
for (L = 1; L <= □; L++)
{
   // D = number of points in the sub DFT
   // butterfly structure at stage L
   D = 1 << L; // D = 4
   // B = number of butterflies in the sub DFT
   // butterfly structure at stage L
   B = D >> 1; // B = 2
   // reset twiddle factor U = W_N^0 = 1

   // j is the butterfly index in the sub DFT
   // butterfly structure
   for (j = 0; j < □; j++) // run 2 times
   {
      // i is butterfly top line index
      for (i = j; i < N; i += □) // run 2 times
      {
         //compute butterfly bottom line index k
         k = i + □;

         // compute butterfly outputs X[i], X[k]
         // X[i] = X[i] + U*X[k]
         // X[k] = X[i] – U*X[k]
      }
      // update twiddle factor U = U*W[L-1]
   }
}
```

L = 2



Reset $U = W_8^0 = 1$

Butterfly Structure (L = 2)

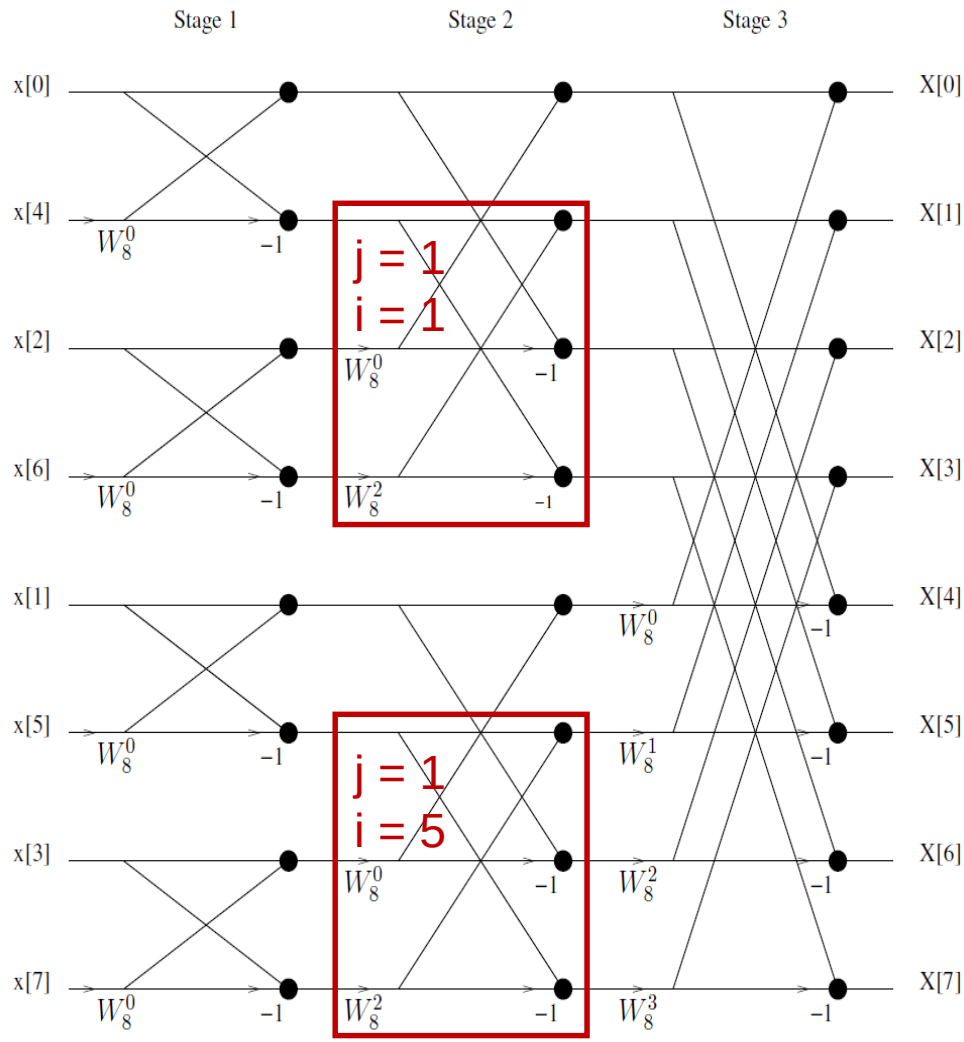
```
// L is the current stage
for (L = 1; L <= □; L++)
{
   // D = number of points in the sub DFT
   // butterfly structure at stage L
   D = 1 << L; // D = 4
   // B = number of butterflies in the sub DFT
   // butterfly structure at stage L
   B = D >> 1; // B = 2
   // reset twiddle factor U = W_N^0 = 1

   // j is the butterfly index in the sub DFT
   // butterfly structure
   for (j = 0; j < □; j++) // run 2 times
   {
      // i is butterfly top line index
      for (i = j; i < N; i += □) // run 2 times
      {
         //compute butterfly bottom line index k
         k = i + □;

         // compute butterfly outputs X[i], X[k]
         // X[i] = X[i] + U*X[k]
         // X[k] = X[i] – U*X[k]
      }
      // update twiddle factor U = U*W[L-1]
   }
}
```

$$W = [W_8^4, W_8^2, W_8^1]$$

L = 2



$U = U*W[1] = W_8^0 * W_8^2 = W_8^2$

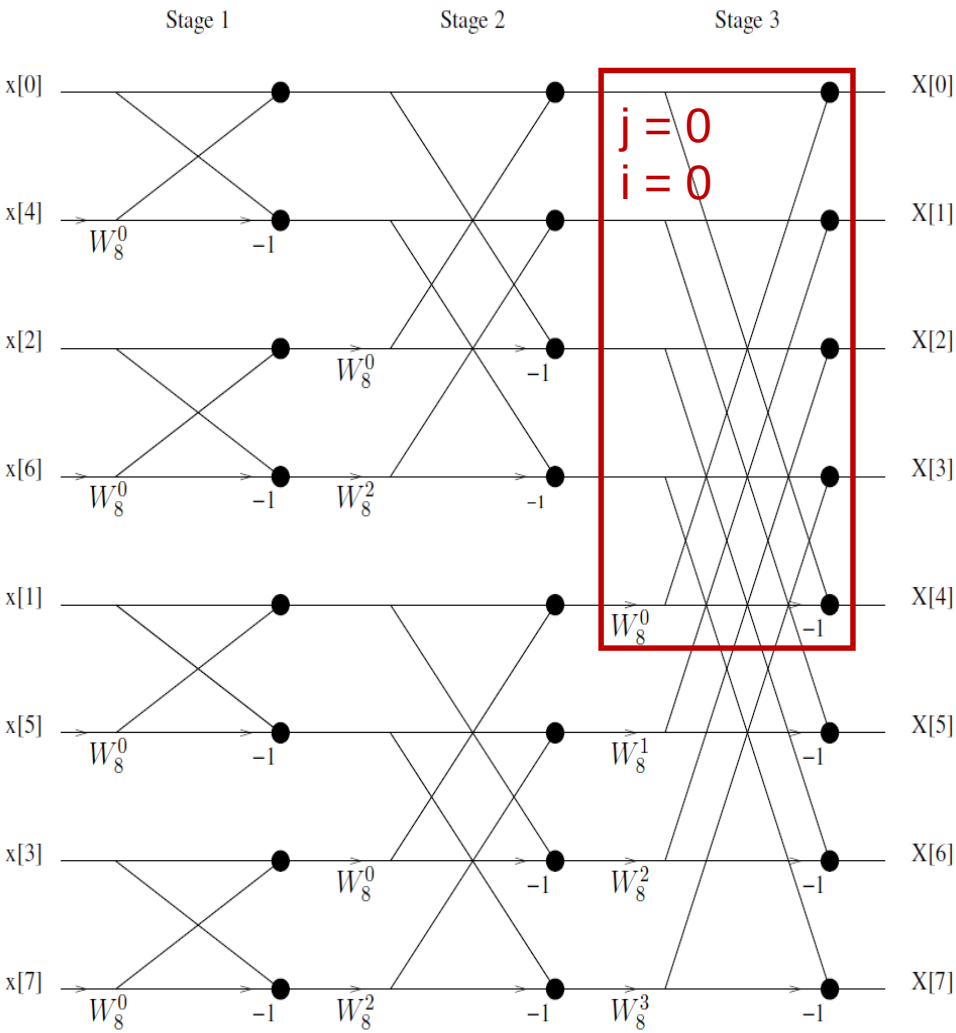
Butterfly Structure (L = 3)

```
// L is the current stage
for (L = 1; L <= □; L++)
{
   // D = number of points in the sub DFT
   // butterfly structure at stage L
   D = 1 << L; // D = 8
   // B = number of butterflies in the sub DFT
   // butterfly structure at stage L
   B = D >> 1; // B = 4
   // reset twiddle factor U = W_N^0 = 1

   // j is the butterfly index in the sub DFT
   // butterfly structure
   for (j = 0; j < □; j++) // run 4 times
   {
      // i is butterfly top line index
      for (i = j; i < N; i += □) // run 1 time
      {
         //compute butterfly bottom line index k
         k = i + □;

         // compute butterfly outputs X[i], X[k]
         // X[i] = X[i] + U*X[k]
         // X[k] = X[i] – U*X[k]
      }
      // update twiddle factor U = U*W[L-1]
   }
}
```

L = 3



Reset U = W_8^0 = 1

Butterfly Structure (L = 3)

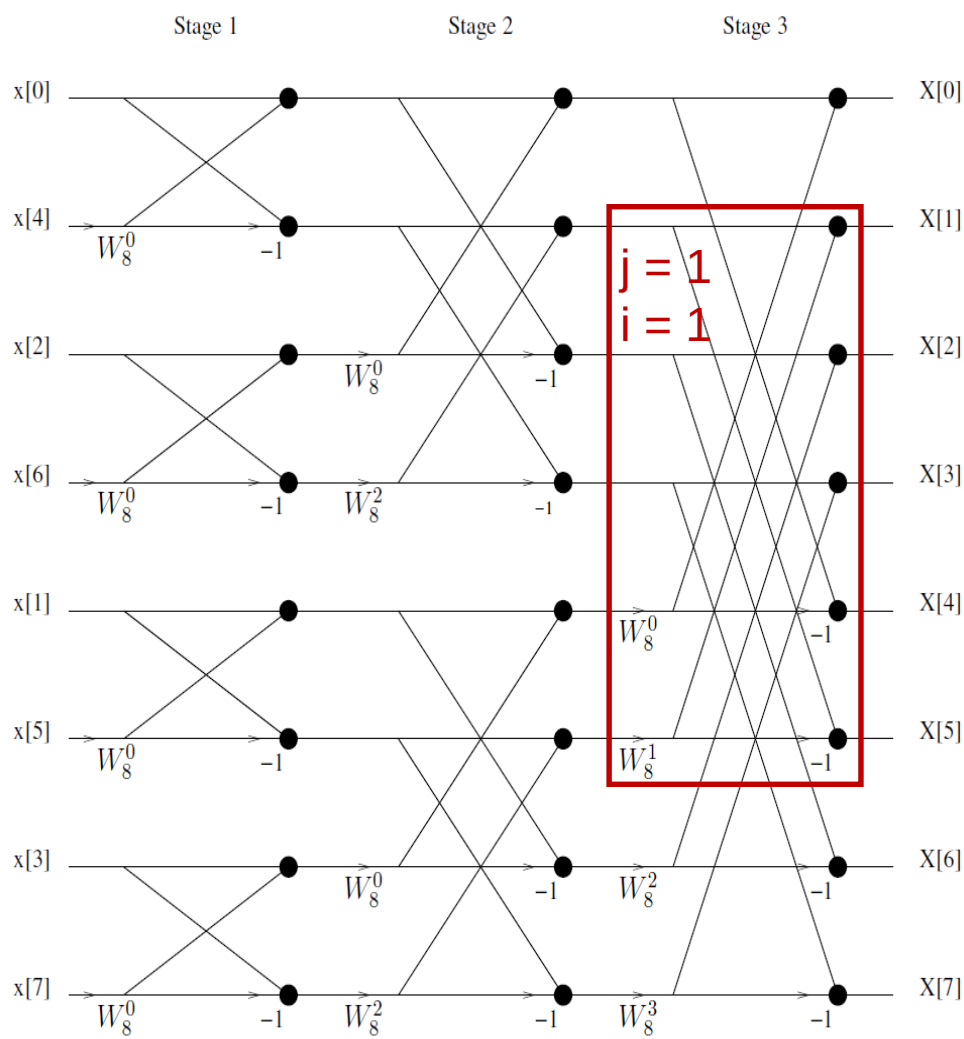
```
// L is the current stage
for (L = 1; L <= □; L++)
{
   // D = number of points in the sub DFT
   // butterfly structure at stage L
   D = 1 << L; // D = 8
   // B = number of butterflies in the sub DFT
   // butterfly structure at stage L
   B = D >> 1; // B = 4
   // reset twiddle factor U = W_N^0 = 1

   // j is the butterfly index in the sub DFT
   // butterfly structure
   for (j = 0; j < □; j++) // run 4 times
   {
      // i is butterfly top line index
      for (i = j; i < N; i += □) // run 1 time
      {
         //compute butterfly bottom line index k
         k = i + □;

         // compute butterfly outputs X[i], X[k]
         // X[i] = X[i] + U*X[k]
         // X[k] = X[i] – U*X[k]
      }
      // update twiddle factor U = U*W[L-1]
   }
}
```

$$W = [W_8^4, W_8^2, W_8^1]$$

L = 3



$U = U*W[2] = W_8^0 * W_8^1 = W_8^1$

Butterfly Structure (L = 3)

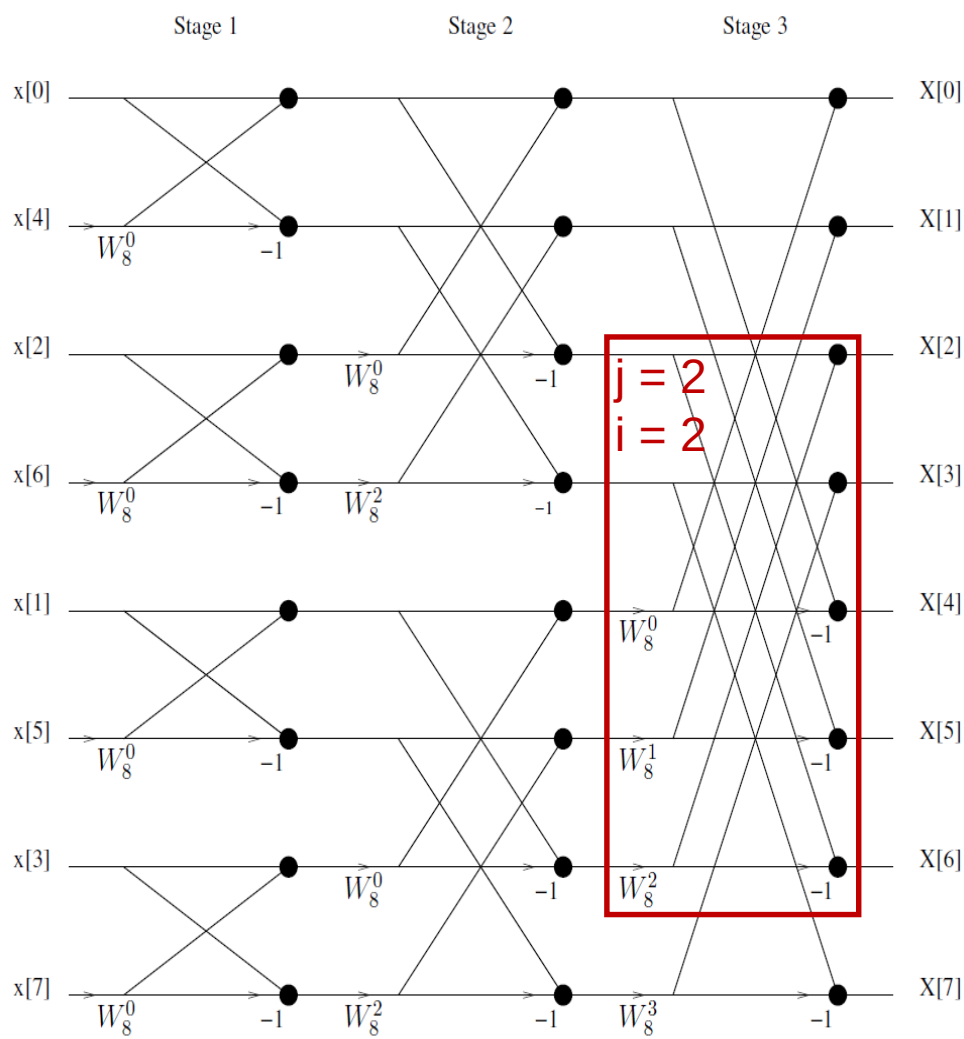
```
// L is the current stage
for (L = 1; L <= □; L++)
{
   // D = number of points in the sub DFT
   // butterfly structure at stage L
   D = 1 << L; // D = 8
   // B = number of butterflies in the sub DFT
   // butterfly structure at stage L
   B = D >> 1; // B = 4
   // reset twiddle factor U = W_N^0 = 1

   // j is the butterfly index in the sub DFT
   // butterfly structure
   for (j = 0; j < □; j++) // run 4 times
   {
      // i is butterfly top line index
      for (i = j; i < N; i += □) // run 1 time
      {
         //compute butterfly bottom line index k
         k = i + □;

         // compute butterfly outputs X[i], X[k]
         // X[i] = X[i] + U*X[k]
         // X[k] = X[i] - U*X[k]
      }
      // update twiddle factor U = U*W[L-1]
   }
}
```

$$W = [W_8^4, W_8^2, W_8^1]$$

L = 3



$U = U*W[2] = W_8^1 * W_8^1 = W_8^2$

Butterfly Structure (L = 3)

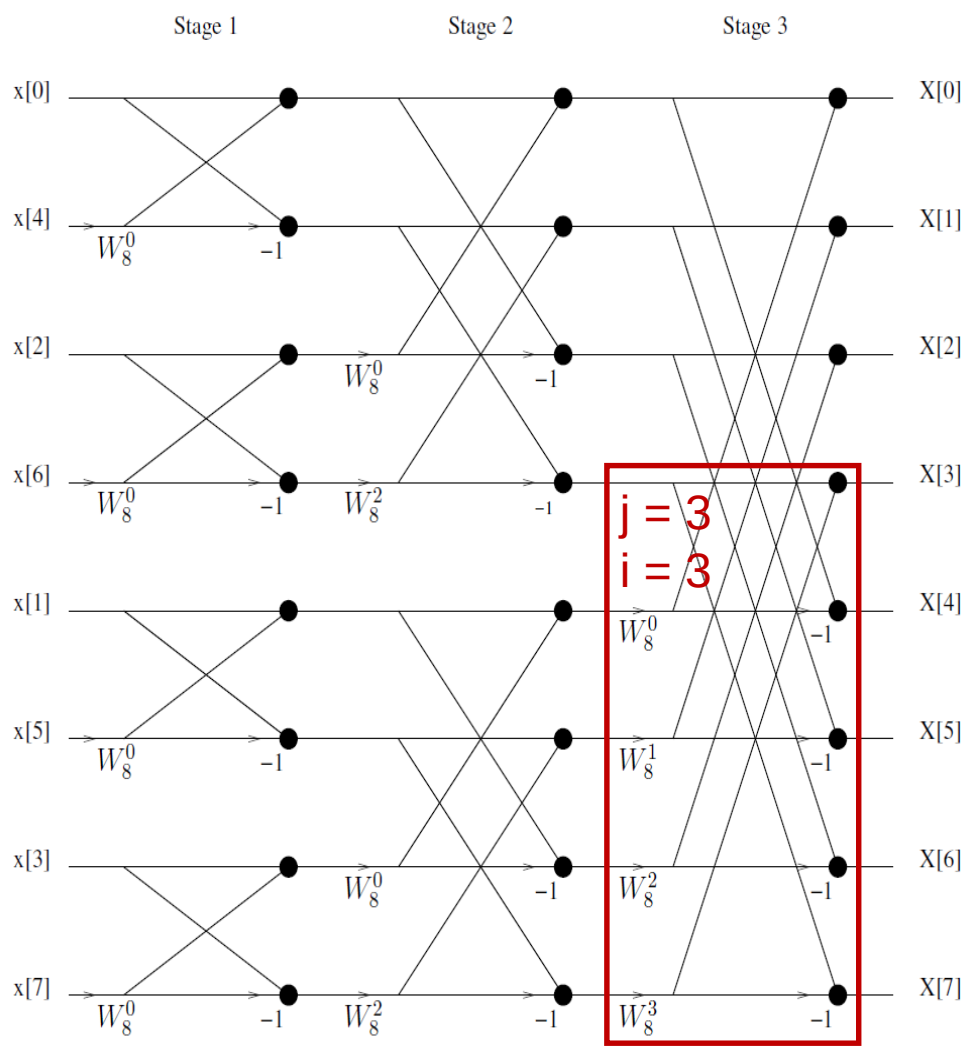
```
// L is the current stage
for (L = 1; L <= □; L++)
{
   // D = number of points in the sub DFT
   // butterfly structure at stage L
   D = 1 << L; // D = 8
   // B = number of butterflies in the sub DFT
   // butterfly structure at stage L
   B = D >> 1; // B = 4
   // reset twiddle factor U = W_N^0 = 1

   // j is the butterfly index in the sub DFT
   // butterfly structure
   for (j = 0; j < □; j++) // run 4 times
   {
      // i is butterfly top line index
      for (i = j; i < N; i += □) // run 1 time
      {
         //compute butterfly bottom line index k
         k = i + □;

         // compute butterfly outputs X[i], X[k]
         // X[i] = X[i] + U*X[k]
         // X[k] = X[i] – U*X[k]
      }
      // update twiddle factor U = U*W[L-1]
   }
}
```

$$W = [W_8^4, W_8^2, W_8^1]$$

L = 3



$U = U*W[2] = W_8^2 * W_8^1 = W_8^3$

Complex Number Arithmetic

Butterfly equation:

- X[i] = X[i] + U*X[k]
- X[k] = X[i] – U*X[k]

Twiddle factor:

- U = U*W[L-1]

These require complex multiplications and additions.

Example:

- temp = temp.re + j*temp.im = U*X[k] = (U.re + j*U.im)*(X[k].re + j*X[k].im)
- temp.re = U.re * X[k].re - U.im * X[k].im
- temp.im = U.re * X[k].im + U.im * X[k].re

Overflow

- **Butterfly equation:**

  X[i] = X[i] + U*X[k]

  X[k] = X[i] – U*X[k]

- **Where can overflow occur?**

- **How to prevent overflow?**

Overflow

To prevent overflow, scale the output at every stage:

X[i] = X[i]*0.5 + U*X[k]*0.5

X[k] = X[i]*0.5 – U*X[k]*0.5

How to restore to the correct DFT?

- Multiply by $2^{log_2 N} = N$

Question

To compute a 1024-point FFT, if scaling of 0.5 is applied at every stage, how to restore the correct DFT result?

a) Multiply the FFT output by 2.

b) Multiply the FFT output by 10.

c) Multiply the FFT output by 1024.

d) Divide the FFT output by 2.

Ira A. Fulton Schools of Engineering

Arizona State University